

Mizar Evo: Design Principles

Connecting automatic proof, readable mathematics, and checked computation

TPP 2026, RIKEN AIP Tokyo Office, November 2026

presenter notes edition

Mizar Evo project

TPP 2026, RIKEN AIP Tokyo Office, November 16–17, 2026

Mizar Evo: Design Principles

Mizar Evo: Design Principles

Connecting automatic proof, readable mathematics, and checked computation
TPP 2026, RIKEN AIP Tokyo Office, November 2026
presenter notes edition

Mizar Evo project

TPP 2026, RIKEN AIP Tokyo Office, November 16–17, 2026

Read aloud:

- **Mizar is a proof checker. It has a fifty-year-old library, written in a readable mathematical language over first-order set theory.**
- **Mizar Evolution, or Mizar Evo, is a redesign of its language and tools. I will start with current Mizar's modernization challenges, then show how the new specification addresses them.**
- Everything I say has a label: fact, research hypothesis, or future direction.

0.2 - How To Read The Claims

deep dive

Level	Meaning	Marked how
fact	existing systems, published benchmarks, the Mizar Evo specification and main branch	no tag
research hypothesis	a claim that Mizar Evo is built to test	tagged in the text
future direction	a goal with no fixed date or design yet	tagged in the text

Code labels follow the Bialystok deck: exact MML excerpt, specification example, sketch.

- The specification and the implementation are different things. One frame near the end separates them.

2026-10-09

Mizar Evo: Design Principles

└ Part 0. Introduction

└ 0.2 - How To Read The Claims

deep dive

0.2 - How To Read The Claims

deep dive

Level	Meaning	Marked how
fact	existing systems, published benchmarks, the Mizar Evo specification and main branch	no tag
research hypothesis	a claim that Mizar Evo is built to test	tagged in the text
future direction	a goal with no fixed date or design yet	tagged in the text

Code labels follow the Bialystok deck: exact MML excerpt, specification example, sketch.

- The specification and the implementation are different things. One frame near the end separates them.

Read aloud:

- Code labels follow the Bialystok deck: exact MML excerpt, specification example, sketch.
- The specification and the implementation are different things. One frame near the end separates them.

0.3 - Current Mizar: Six Challenges And Design Principles

Challenge in modernizing Mizar	New design principle
§1 Foundation: retain logic and MML; renew tools	First-order logic, set theory, and a small kernel
§2 Writing: implicit types, registrations, overloads	Mathematical abstraction; explicit, traceable choices
§3 Generics: separate definition and scheme mechanisms	Templates for definitions, theorems, and schemes
§4 Computation: connect proofs and procedures	Contracts, invariants, and termination checks
§5 Development: tools for a large library	Modules, namespaces, packages, incremental builds, LSP
§6 Checking: external search and evidence	ATP flow; derive formula instances; trusted SAT check

Mizar Evo: Design Principles

└ Part 0. Introduction

└ 0.3 - Current Mizar: Six Challenges And Design Principles

0.3 - Current Mizar: Six Challenges And Design Principles

Challenge in modernizing Mizar	New design principle
\$1 Foundation: retain logic and MML; renew tools	First-order logic, set theory, and a small kernel
\$2 Writing: implicit types, registrations, overloads	Mathematical abstraction; explicit, traceable choices
\$3 Generics: separate definition and scheme mechanisms	Templates for definitions, theorems, and schemes
\$4 Computation: connect proofs and procedures	Contracts, invariants, and termination checks
\$5 Development: tools for a large library	Modules, namespaces, packages, incremental builds, LSP
\$6 Checking: external search and evidence	ATP flow; derive formula instances; trusted SAT check

Optional detail (see slide).

Optional notes and sources:

- Each row pairs a challenge with its design principle. Sections 1–6 follow these rows. First-order logic and readable mathematics are assets to preserve.
- The closing status frame separates the specification from implementation.
- Source: specification 01, 18, 20, 21, 23; Bialystok problem-driven stories.

1.1 - Keep The Foundation, Rebuild The Tools

*Keep the mathematical layer.
Rebuild the tools under it and around it.*

- Mizar Evo keeps first-order logic and Tarski-Grothendieck set theory as the base logic.
- It keeps soft types, modes, attributes, registrations, structures, and declarative proofs.
- **Separate proof search from checking; a small kernel decides acceptance.**
Details in section 6.

Mizar Evo: Design Principles

└ Part 1. Logical Foundation

└ 1.1 - Keep The Foundation, Rebuild The Tools

1.1 - Keep The Foundation, Rebuild The Tools

*Keep the mathematical layer.
Rebuild the tools under it and around it.*

- Mizar Evo keeps first-order logic and Tarski-Grothendieck set theory as the base logic.
- It keeps soft types, modes, attributes, registrations, structures, and declarative proofs.
- Separate proof search from checking; a small kernel decides acceptance. Details in section 6.

Read aloud:

- **Keep the mathematical layer. Rebuild the tools under it and around it.**
- **Mizar Evo keeps first-order logic and Tarski-Grothendieck set theory as the base logic.**
- It keeps soft types, modes, attributes, registrations, structures, and declarative proofs.
- **Separate proof search from checking; a small kernel decides acceptance. Details in section 6.**

Optional notes and sources:

- The shared structure concerns mathematical writing and proof tools, not identical logics or representations.
- Neither success rates nor reconstruction failures show a first-order advantage. Evaluate ease of writing, automation, and proof checking.

1.2 - Current Mizar: Functions In Set Theory

Function application is a defined set-theoretic relation **exact MML excerpt**

```
func f.x -> set means  
:Def2:  
[x,it] in f if x in dom f otherwise it = {};
```

A typed function is a soft type over partial functions **exact MML excerpt**

```
definition  
  let X,Y;  
  mode Function of X,Y is quasi_total PartFunc of X,Y;  
end;
```

- Here a function is an ordinary first-order object: a set of pairs. Application is defined, not built into the logic.
- Nothing in the base logic has to be higher-order.
- The price: if you write raw set theory, the domain, the graph, functionality, and membership all appear in the text.

Mizar Evo: Design Principles

└ Part 1. Logical Foundation

└ 1.2 - Current Mizar: Functions In Set Theory

1.2 - Current Mizar: Functions In Set Theory

Function application is a defined set-theoretic relation exact MML excerpt

```
func f.x -> set means
:Def2:
[x,it] in f if x in dom f otherwise it = {};
```

A typed function is a soft type over partial functions exact MML excerpt

```
definition
let X,Y;
mode Function of X,Y is quasi_total PartFunc of X,Y;
end;
```

- Here a function is an ordinary first-order object: a set of pairs. Application is defined, not built into the logic.
- Nothing in the base logic has to be higher-order.
- The price: if you write raw set theory, the domain, the graph, functionality, and membership all appear in the text.

Read aloud:

- Function application is a defined set-theoretic relation.
- A typed function is a soft type over partial functions.
- **Here a function is an ordinary first-order object: a set of pairs. Application is defined, not built into the logic.**
- **Nothing in the base logic has to be higher-order.**
- The price: if you write raw set theory, the domain, the graph, functionality, and membership all appear in the text.

Optional notes and sources:

- Source: current MML `funct_1.miz`, lines 138-140 (`FUNCT_1: def 2`), and `funct_2.miz`, lines 87-90 (checked October 7, 2026). URLs:
https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz,
https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz. GPL-3.0-or-later / CC-BY-SA-3.0-or-later.
- `quasi_total` (`FUNCT_2`, lines 36-44) says the domain is all of X unless Y is empty.

2.1 - Preserve And Extend Mathematical Writing

What the author actually writes **sketch** (valid in current Mizar and Mizar Evo)

```
let X, Y be set;  
let f be Function of X, Y;  
let x be Element of X;  
... f.x ...
```

- It is the same set-theoretic function. But the author writes Function of X,Y and f.x, not pairs and domains.
- Soft types, modes, attributes, registrations, schemes, and declarative proofs are not just a nicer way to write the same thing. They are the language design that lifts first-order set theory up to readable mathematics.
- New specification: preserve and extend mathematical writing; make implicit type, registration, and overload choices explicit and traceable.

Mizar Evo: Design Principles

└ Part 2. Readable Mathematics

└ 2.1 - Preserve And Extend Mathematical Writing

2.1 - Preserve And Extend Mathematical Writing

What the author actually writes **sketch** (valid in current Mizar and Mizar Evo)

```
let X, Y be set;  
let f be Function of X, Y;  
let x be Element of X;  
... f.x ...
```

- It is the same set-theoretic function. But the author writes Function of X,Y and f.x, not pairs and domains.
- Soft types, modes, attributes, registrations, schemes, and declarative proofs are not just a nicer way to write the same thing. They are the language design that lifts first-order set theory up to readable mathematics.
- New specification: preserve and extend mathematical writing; make implicit type, registration, and overload choices explicit and traceable.

Read aloud:

- What the author actually writes.
- **It is the same set-theoretic function. But the author writes Function of X,Y and f.x, not pairs and domains.**
- **Soft types, modes, attributes, registrations, schemes, and declarative proofs are not just a nicer way to write the same thing. They are the language design that lifts first-order set theory up to readable mathematics.**
- **New specification: preserve and extend mathematical writing; make implicit type, registration, and overload choices explicit and traceable.**

2.2 - Make Operation Choices Explicit And Traceable

Select one of a ring's two operation views **sketch**

```
let R be commutative Ring;  
f(R);                :: ambiguous Magma view  
f(R qua AddMagma);   :: use addition  
f(R qua MulMagma);   :: use multiplication
```

- **Challenge:** a ring has additive and multiplicative Magma views. $f(R)$ is ambiguous.
- **New specification:** `qua` explicitly selects the additive or multiplicative view.
- Trace automatically applied registrations and their derivation paths.

Mizar Evo: Design Principles

└ Part 2. Readable Mathematics

└ 2.2 - Make Operation Choices Explicit And Traceable

2.2 - Make Operation Choices Explicit And Traceable

Select one of a ring's two operation views sketch

```

let R be commutative Ring;
f(R);
f(R qua AddMagma);
f(R qua MulMagma);

```

... :: ambiguous Magma view
... :: use addition
... :: use multiplication

- Challenge: a ring has additive and multiplicative Magma views. $f(R)$ is ambiguous.
- New specification: qua explicitly selects the additive or multiplicative view.
- Trace automatically applied registrations and their derivation paths.

Read aloud:

- Select one of a ring's two operation views.
- **Challenge: a ring has additive and multiplicative Magma views. $f(R)$ is ambiguous.**
- **New specification: qua explicitly selects the additive or multiplicative view.**
- Trace automatically applied registrations and their derivation paths.

Optional notes and sources:

- f uses a Magma operation; definitions, inheritance, and registrations are assumed.
- qua already exists in Mizar; this sketch illustrates explicit inheritance-path selection.
- Source: doc/spec/en/19.overload_resolution.md section 19.3.1; 17.clusters_and_registrations.md, Traceability.

3.1 - Templates: Share Functor Definitions And Result Types

Example: the sum of functions. Add their values at each point.

Generic functor **sketch** (existence and uniqueness proofs omitted)

```
definition
  let T be type extends non empty AddMagma;
  let I be non empty set;
  let f, g be Function of I, T;
  func AddDef: Add[T,I](f,g) -> Function of I,T means
    for i being Element of I holds it.i = T.add(f.i,g.i);
end;
```

- **Current MML: real and complex result types need separate redefinitions and registrations.**
- **New specification: parameterize the value type T to share the definition and result type.**

Mizar Evo: Design Principles

└ Part 3. Generic Mathematics

└ 3.1 - Templates: Share Functor Definitions And Result Types

3.1 - Templates: Share Functor Definitions And Result Types

Example: the sum of functions. Add their values at each point.

Generic functor **sketch** (existence and uniqueness proofs omitted)

```

definition
  let T be type extends non empty AddMagma;
  let I be non empty set;
  let f, g be Function of I, T;
  func AddDef: Add[I,I](f,g) -> Function of I,T means
    for i being Element of I holds it.1 = T.add(f.i,g.i);
end;

```

- Current MML: real and complex result types need separate redefinitions and registrations.
- New specification: parameterize the value type T to share the definition and result type.

Read aloud:

- **Example: the sum of functions. Add their values at each point.**
- Generic functor.
- **Current MML: real and complex result types need separate redefinitions and registrations.**
- **New specification: parameterize the value type T to share the definition and result type.**

Optional notes and sources:

- T supplies addition; I is a common nonempty domain. Suitable structures and registrations are assumed.
- Source: doc/spec/en/18.templates.md sections 18.2.2, 18.7, 18.10.1; sample_codes.md AddMagma; MML VALUED_1 (https://mizar.uwb.edu.pl/version/current/html/valued_1.html).

Induction as a theorem with a predicate parameter specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

- A predicate parameter gives a family of first-order theorems, one for each predicate you put in.
- Instantiation is explicit: by `NatInduction[P]`, `Base`, `Step` after a `defpred`.
- Functor parameters are schema-level symbols, not sets. This keeps the logic first-order.
- Details: Bialystok deck, Story 6 (`PermProduct [T]`, `qua views`), and Backup 4 here.

Mizar Evo: Design Principles

└ Part 3. Generic Mathematics

└ 3.2 - Schemes Become Ordinary Templates

deep dive

Induction as a theorem with a predicate parameter **specification example**

```

definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;

```

- A predicate parameter gives a family of first-order theorems, one for each predicate you put in.
- Instantiation is explicit: by `NatInduction[P]`, `Base`, `Step` after a `defpred`.
- Functor parameters are schema-level symbols, not sets. This keeps the logic first-order.
- Details: Bialystok deck, Story 6 (`PermProduct [T]`, `qua views`), and Backup 4 [here](#).

Read aloud:

- Induction as a theorem with a predicate parameter.
- A predicate parameter gives a family of first-order theorems, one for each predicate you put in.
- Instantiation is explicit: by `NatInduction[P]`, `Base`, `Step` after a `defpred`.
- Functor parameters are schema-level symbols, not sets. This keeps the logic first-order.
- Details: Bialystok deck, Story 6 (`PermProduct [T]`, `qua views`), and Backup 4 [here](#).

4.1 - Computation: Verified Algorithms

- Mizar uses declarative proofs and built-in automation. It has no user-programmable tactic language.
- **Challenge: connect mathematical proofs and executable computation in one checking framework.**
- **New specification: an algorithm is a procedure with a contract. Its correctness, invariants, and termination are checked.**
- Uses: verify algorithms themselves, and use verified procedures to support proofs.

Mizar Evo: Design Principles

└ Part 4. Verified Computation

└ 4.1 - Computation: Verified Algorithms

4.1 - Computation: Verified Algorithms

- Mizar uses declarative proofs and built-in automation. It has no user-programmable tactic language.
- **Challenge: connect mathematical proofs and executable computation in one checking framework.**
- **New specification: an algorithm is a procedure with a contract. Its correctness, invariants, and termination are checked.**
- Uses: verify algorithms themselves, and use verified procedures to support proofs.

Read aloud:

- Mizar uses declarative proofs and built-in automation. It has no user-programmable tactic language.
- **Challenge: connect mathematical proofs and executable computation in one checking framework.**
- **New specification: an algorithm is a procedure with a contract. Its correctness, invariants, and termination are checked.**
- Uses: verify algorithms themselves, and use verified procedures to support proofs.

Optional notes and sources:

- Source: `doc/spec/en/20.algorithm_and_verification.md`, sections 20.1 and 20.12.

4.2 - Algorithms: Contracts, Proofs, Computation

Euclid's algorithm with a contract **specification example**

```
terminating algorithm euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a;  var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y);  decreasing y;
    const r := x mod y;  x := y;  y := r;
  end;
  return x;
end;
```

- Contracts, invariants, and termination measures become first-order proof obligations, checked like theorems.
- Two roles: a verified automation procedure, and a checked computation run by by computation. Status: specified. MVM execution and code extraction come later.

Mizar Evo: Design Principles

└ Part 4. Verified Computation

└ 4.2 - Algorithms: Contracts, Proofs, Computation

4.2 - Algorithms: Contracts, Proofs, Computation

Euclid's algorithm with a contract **specification example**

```

terminating algorithm euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a; var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y); decreasing y;
    const r := x mod y; x := y; y := r;
  end
  return x;
end;

```

- Contracts, invariants, and termination measures become first-order proof obligations, checked like theorems.
- Two roles: a verified automation procedure, and a checked computation run by by computation. Status: specified. MVM execution and code extraction come later.

Read aloud:

- **Euclid's algorithm with a contract.**
- **Contracts, invariants, and termination measures become first-order proof obligations, checked like theorems.**
- **Two roles: a verified automation procedure, and a checked computation run by by computation. Status: specified. MVM execution and code extraction come later.**

Optional notes and sources:

- Source: doc/spec/en/20.algorithm_and_verification.md, section 20.12 (condensed: the enclosing definition block and let a, b be Nat; are omitted).

Future direction, not current capability:

- number-theoretic and combinatorial algorithms; symbolic algorithms; optimization procedures;
- later: cryptographic algorithms and protocols; quantum and hybrid classical-quantum algorithms.

One workflow for all of them:

- 1 write the procedure; 2. state the contract; 3. give invariants and termination; 4. generate proof obligations; 5. prove them with ATPs and check them in the kernel; 6. run on concrete inputs; 7. extract code later.
- **Algorithms make the gap between tactics and application algorithms smaller. The targets above are future directions.**

Mizar Evo: Design Principles

└ Part 4. Verified Computation

└ 4.3 - Future Targets For Algorithms

deep dive

4.3 - Future Targets For Algorithms

deep dive

Future direction, not current capability:

- number-theoretic and combinatorial algorithms; symbolic algorithms; optimization procedures;
- later: cryptographic algorithms and protocols; quantum and hybrid classical-quantum algorithms.

One workflow for all of them:

- write the procedure; 2. state the contract; 3. give invariants and termination; 4. generate proof obligations; 5. prove them with ATPs and check them in the kernel; 6. run on concrete inputs; 7. extract code later.
- **Algorithms make the gap between tactics and application algorithms smaller. The targets above are future directions.**

Read aloud:

- number-theoretic and combinatorial algorithms; symbolic algorithms; optimization procedures;
- later: cryptographic algorithms and protocols; quantum and hybrid classical-quantum algorithms.
- write the procedure; 2. state the contract; 3. give invariants and termination; 4. generate proof obligations; 5. prove them with ATPs and check them in the kernel; 6. run on concrete inputs; 7. extract code later.
- **Algorithms make the gap between tactics and application algorithms smaller. The targets above are future directions.**

5.1 - Development Problems And The New Design

Seen after fifty years of MML	Mizar Evo
the article as the unit of dependency	modules with explicit imports
global name management	namespaces, fully qualified names
weak distribution and versioning	packages, SemVer, lock files
full rebuilds	incremental builds with dependency fingerprints
ATP as an add-on, automation hard to see	first-class ATP pipeline, resolution traces, kernel evidence
weak IDE and machine-readable I/O	LSP, structured diagnostics, agent interface

- **Keeping Mizar's mathematical ideas and keeping 1970s software architecture are two different things.**
- **Of course, we also modernize the whole development infrastructure. The Bialystok deck has the details. Here one table is enough.**

Mizar Evo: Design Principles

└ Part 5. Development Infrastructure

└ 5.1 - Development Problems And The New Design

5.1 - Development Problems And The New Design

Seen after fifty years of MML	Mizar Evo
the article as the unit of dependency	modules with explicit imports
global name management	namespaces, fully qualified names
weak distribution and versioning	packages, SemVer, lock files
full rebuilds	incremental builds with dependency fingerprints
ATP as an add-on, automation hard to see	first-class ATP pipeline, resolution traces, kernel evidence
weak IDE and machine-readable I/O	LSP, structured diagnostics, agent interface

- Keeping Mizar's mathematical ideas and keeping 1970s software architecture are two different things.
- Of course, we also modernize the whole development infrastructure. The Bialystok deck has the details. Here one table is enough.

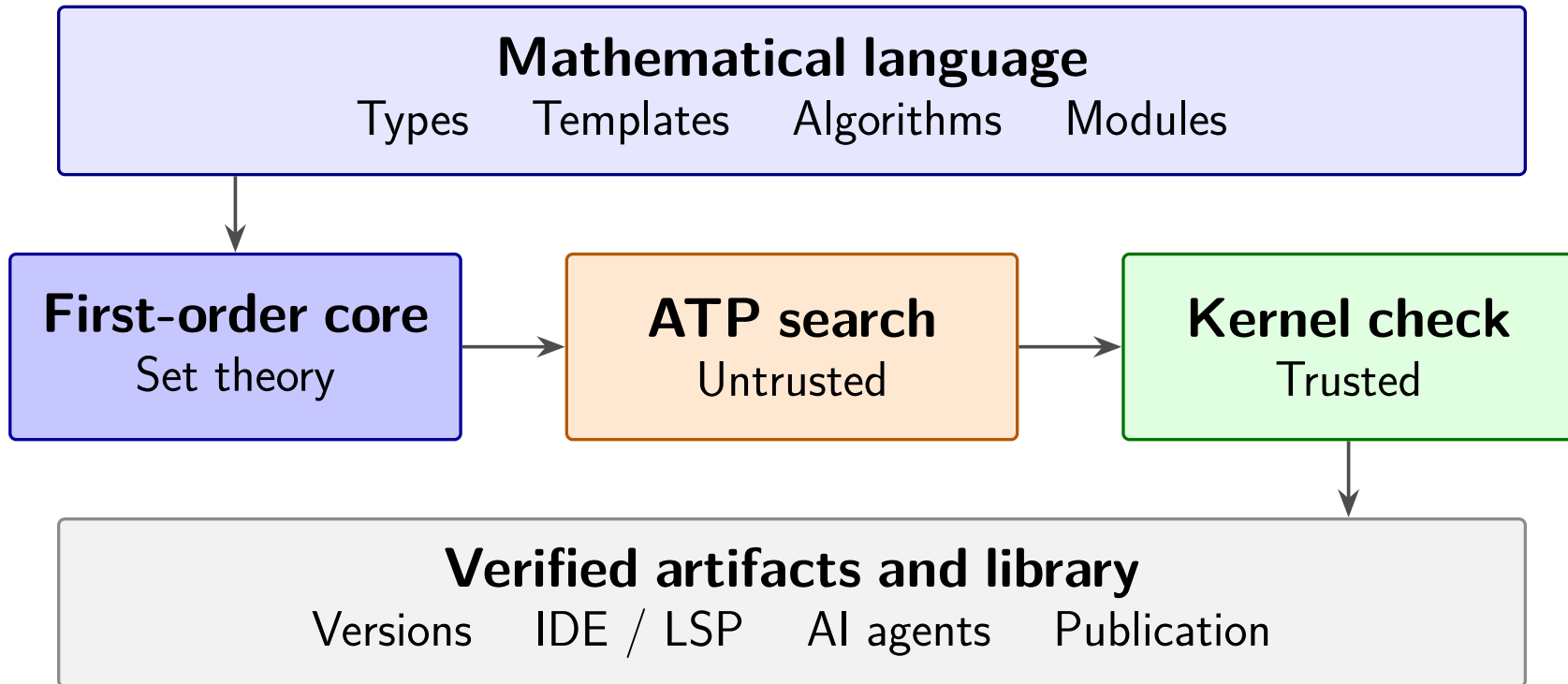
Read aloud:

- **Keeping Mizar's mathematical ideas and keeping 1970s software architecture are two different things.**
- **Of course, we also modernize the whole development infrastructure. The Bialystok deck has the details. Here one table is enough.**

Optional notes and sources:

- This is not a criticism. It is software engineering that was not common fifty years ago, brought into a formal mathematics environment.
- Details: Bialystok deck, Stories 1, 3, 5, 8; Backups 6-8 here.

5.2 - The Whole Picture



- Rich mathematics, a small first-order core, and modern infrastructure.

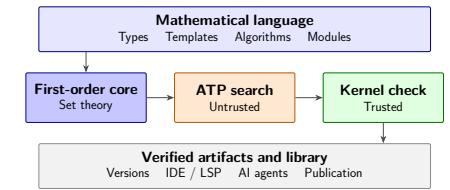
2026-10-09

Mizar Evo: Design Principles

└ Part 5. Development Infrastructure

└ 5.2 - The Whole Picture

5.2 - The Whole Picture



• Rich mathematics, a small first-order core, and modern infrastructure.

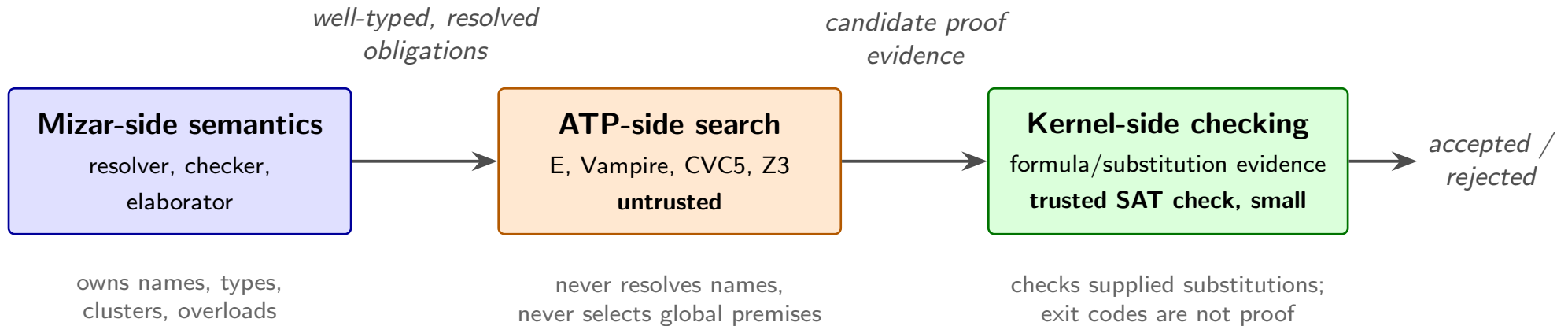
Read aloud:

- **Rich mathematics, a small first-order core, and modern infrastructure.**

Optional notes and sources:

- Elaboration maps the mathematical language to a first-order representation. The kernel checks evidence proposed by external ATPs. Verified artifacts and the library support IDE / LSP, AI, and publication.

6.1 - Automation: Separate Search From Checking



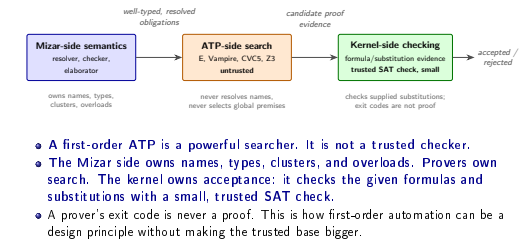
- A first-order ATP is a powerful searcher. It is not a trusted checker.
- The Mizar side owns names, types, clusters, and overloads. Provers own search. The kernel owns acceptance: it checks the given formulas and substitutions with a small, trusted SAT check.
- A prover's exit code is never a proof. This is how first-order automation can be a design principle without making the trusted base bigger.

Mizar Evo: Design Principles

└ Part 6. Checking And Automation

└ 6.1 - Automation: Separate Search From Checking

6.1 - Automation: Separate Search From Checking



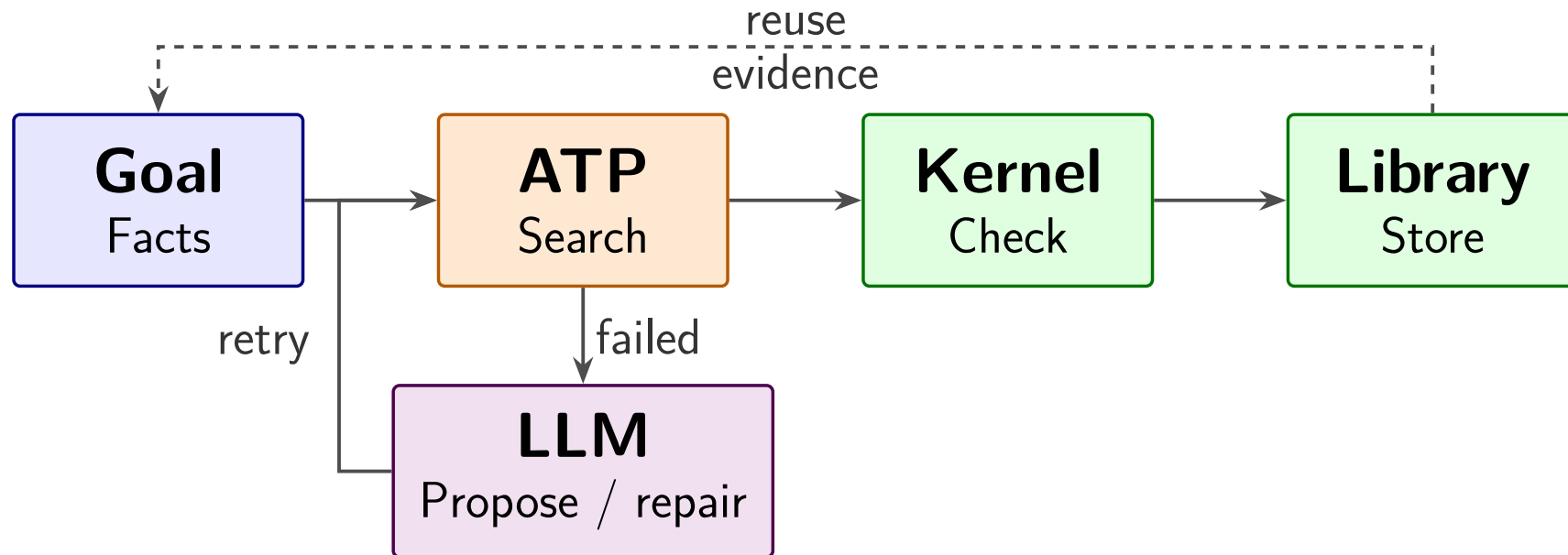
Read aloud:

- **A first-order ATP is a powerful searcher. It is not a trusted checker.**
- **The Mizar side owns names, types, clusters, and overloads. Provers own search. The kernel owns acceptance: it checks the given formulas and substitutions with a small, trusted SAT check.**
- A prover's exit code is never a proof. This is how first-order automation can be a design principle without making the trusted base bigger.

Optional notes and sources:

- Source: `doc/design/architecture/en/08.reasoning_boundary.md`; Bialystok deck, Story 4; Backup 5 here shows what the evidence contains.

6.2 - Using ATPs: Search, Check, And Reuse



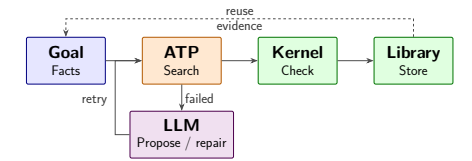
- Goal and facts → ATP search → kernel check → store in the library and reuse.
- For open goals, revise lemmas or the plan and retry. The figure shows design intent.

Mizar Evo: Design Principles

└ Part 6. Checking And Automation

└ 6.2 - Using ATPs: Search, Check, And Reuse

6.2 - Using ATPs: Search, Check, And Reuse



- Goal and facts → ATP search → kernel check → store in the library and reuse.
- For open goals, revise lemmas or the plan and retry. The figure shows design intent.

Read aloud:

- **Goal and facts → ATP search → kernel check → store in the library and reuse.**
- For open goals, revise lemmas or the plan and retry. The figure shows design intent.

Optional notes and sources:

- LLMs can help propose and repair. Acceptance depends on the evidence check, explained on the next frame.
- Current ATP input is cited premises and local hypotheses. Whole-library premise selection and the loop's cost-effectiveness remain evaluation targets.
- Source: `doc/spec/en/21.source_code_annotation_and_atp.md` section 21.7.2;
`doc/design/architecture/en/21.ai_agent_interface.md`.

6.3 - From Evidence To Instances To SAT Checking

Evidence → Formula instances → SAT check

- ① **Evidence:** source formulas, explicit substitutions, provenance, and the target goal.
- ② **Kernel:** check the bindings and substitutions, then derive the formula instances.
- ③ **SAT:** encode the instances and the negated goal. Accept only after the trusted checker confirms UNSAT.

Mizar Evo: Design Principles

└ Part 6. Checking And Automation

└ 6.3 - From Evidence To Instances To SAT Checking

Evidence → Formula instances → SAT check

- **Evidence**: source formulas, explicit substitutions, provenance, and the target goal.
- **Kernel**: check the bindings and substitutions, then derive the formula instances.
- **SAT**: encode the instances and the negated goal. Accept only after the trusted checker confirms UNSAT.

Read aloud:

- **Evidence → Formula instances → SAT check**
- **Evidence**: source formulas, explicit substitutions, provenance, and the target goal.
- **Kernel**: check the bindings and substitutions, then derive the formula instances.
- **SAT**: encode the instances and the negated goal. Accept only after the trusted checker confirms UNSAT.

Optional notes and sources:

- UNSAT establishes the goal from the premises. The kernel derives the instances and SAT problem itself.
- Backend traces and exit codes are diagnostic only. Full evidence fields: Backup 5.
- Source: `doc/design/architecture/en/08.reasoning_boundary.md`,
`15.kernel_certificate_format.md`.

Where The Project Stands (October 2026)

- **Specification:** 24 chapters plus appendices. English is the canonical language.
- **Implemented on the main branch:** the Rust frontend (lexer, parser, syntax tree); name resolution and type checking on an alpha corpus; proof-obligation generation with deterministic discharge; ATP problem encoding and candidate evidence; SAT-backed kernel evidence checking; cache, fingerprint, and build-scheduling milestones.
- **In progress:** end-to-end integration from source to verified artifact; the LSP server; the documentation generator.
- **Planned for later:** MVM execution, code extraction, library-wide premise selection, MML migration.
- This talk does not claim any end-to-end results with external provers.

Mizar Evo: Design Principles

└ Status And Roadmap

└ Where The Project Stands (October 2026)

Where The Project Stands (October 2026)

- Specification: 24 chapters plus appendices. English is the canonical language.
- Implemented on the main branch: the Rust frontend (lexer, parser, syntax tree); name resolution and type checking on an alpha corpus; proof-obligation generation with deterministic discharge; ATP problem encoding and candidate evidence; SAT-backed kernel evidence checking; cache, fingerprint, and build-scheduling milestones.
- In progress: end-to-end integration from source to verified artifact; the LSP server; the documentation generator.
- Planned for later: MVM execution, code extraction, library-wide premise selection, MML migration.
- This talk does not claim any end-to-end results with external provers.

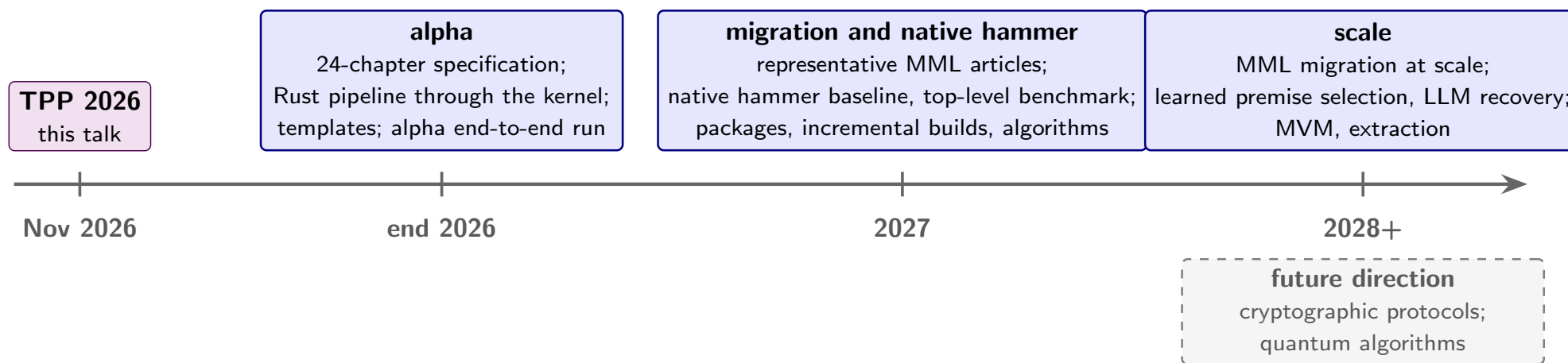
Read aloud:

- **Specification: 24 chapters plus appendices. English is the canonical language.**
- **Implemented on the main branch: the Rust frontend (lexer, parser, syntax tree); name resolution and type checking on an alpha corpus; proof-obligation generation with deterministic discharge; ATP problem encoding and candidate evidence; SAT-backed kernel evidence checking; cache, fingerprint, and build-scheduling milestones.**
- **In progress: end-to-end integration from source to verified artifact; the LSP server; the documentation generator.**
- **Planned for later: MVM execution, code extraction, library-wide premise selection, MML migration.**
- This talk does not claim any end-to-end results with external provers.

Optional notes and sources:

- Source: `doc/design/todo.md`, Crate Status, as of October 7, 2026. Re-check before the talk.

Roadmap



- **2026**: finish the specification, the Rust pipeline through the kernel, template processing, and an alpha end-to-end run.
- **2027**: migrate representative MML articles, build a native hammer baseline, and benchmark on top-level theorems, in the same units MizAR uses.
- **2028 and later**: scale up the migration; learned premise selection; LLM-assisted failure recovery; MVM and extraction. Cryptographic and quantum targets stay future directions.

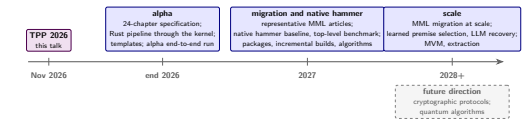
2026-10-09

Mizar Evo: Design Principles

└ Status And Roadmap

└ Roadmap

Roadmap



- 2026: finish the specification, the Rust pipeline through the kernel, template processing, and an alpha end-to-end run.
- 2027: migrate representative MML articles, build a native hammer baseline, and benchmark on top-level theorems, in the same units MizAR uses.
- 2028 and later: scale up the migration; learned premise selection; LLM-assisted failure recovery; MVM and extraction. Cryptographic and quantum targets stay future directions.

Read aloud:

- **2026: finish the specification, the Rust pipeline through the kernel, template processing, and an alpha end-to-end run.**
- **2027: migrate representative MML articles, build a native hammer baseline, and benchmark on top-level theorems, in the same units MizAR uses.**
- 2028 and later: scale up the migration; learned premise selection; LLM-assisted failure recovery; MVM and extraction. Cryptographic and quantum targets stay future directions.

*Keep the foundation small.
Keep the mathematics readable.
Modernize everything else.*

- Thank you. I welcome your questions, especially on MML migration, usability, and how to evaluate automation.

2026-10-09

Mizar Evo: Design Principles

└─ Status And Roadmap

└─ Closing

Closing

*Keep the foundation small.
Keep the mathematics readable.
Modernize everything else.*

- Thank you. I welcome your questions, especially on MML migration, usability, and how to evaluate automation.

Read aloud:

- **Keep the foundation small. Keep the mathematics readable. Modernize everything else.**
- **Thank you. I welcome your questions, especially on MML migration, usability, and how to evaluate automation.**

Backup 1. MizAR 60 In Detail

Source: Jakubův et al., MizAR 60 for Mizar 50, ITP 2023.

- Dataset: MML 1147 exported by MPTP, 57,897 theorems including unnamed top-level lemmas; the same version as the MizAR 40 evaluation, so the results can be compared.
- Holdout: 1,690 / 2,896 (58.4%), without user premise help, 420 CPU s. Training/development/holdout: 90:5:5 (MizAR 40: about 40.6%).
- Over 75% proved when the premises can be chosen from the library by a human or a machine (MizAR 40: 56%).
- Strongest single method: 40% in 30 s in hammering mode; 60% in 120 s with human premises.
- Transfer: the strongest method also works on 13,370 new theorems from 242 new articles in MML 1382.
- Methods: E and Vampire with ENIGMA and Deepire guidance, learned premise selection, and a loop that trains on millions of ATP proofs.

Mizar Evo: Design Principles

└ Backup 1. MizAR 60 In Detail

└ Backup 1. MizAR 60 In Detail

Backup 1. MizAR 60 In Detail

Source: Jakubův et al., MizAR 60 for Mizar 50, ITP 2023.

- Dataset: MML 1147 exported by MPTP, 57,897 theorems including unnamed top-level lemmas; the same version as the MizAR 40 evaluation, so the results can be compared.
- Holdout: 1,690 / 2,896 (58.4%), without user premise help, 420 CPU s. Training/development/holdout: 90:5:5 (MizAR 40: about 40.6%).
- Over 75% proved when the premises can be chosen from the library by a human or a machine (MizAR 40: 56%).
- Strongest single method: 40% in 30 s in hammering mode; 60% in 120 s with human premises.
- Transfer: the strongest method also works on 13,370 new theorems from 242 new articles in MML 1382.
- Methods: E and Vampire with ENIGMA and Deepire guidance, learned premise selection, and a loop that trains on millions of ATP proofs.

Read aloud:

- Dataset: MML 1147 exported by MPTP, 57,897 theorems including unnamed top-level lemmas; the same version as the MizAR 40 evaluation, so the results can be compared.
- Holdout: 1,690 / 2,896 (58.4%), without user premise help, 420 CPU s. Training/development/holdout: 90:5:5 (MizAR 40: about 40.6%).
- Over 75% proved when the premises can be chosen from the library by a human or a machine (MizAR 40: 56%).
- Strongest single method: 40% in 30 s in hammering mode; 60% in 120 s with human premises.
- Transfer: the strongest method also works on 13,370 new theorems from 242 new articles in MML 1382.
- Methods: E and Vampire with ENIGMA and Deepire guidance, learned premise selection, and a loop that trains on millions of ATP proofs.

Optional notes and sources:

- Source: Jakubův et al., MizAR 60 for Mizar 50, ITP 2023.

Backup 2. Sledgehammer Evaluations In Detail

Evaluation	Dataset and method	Success
ITP 2022	5,000 AFP goals, 16 greedy configurations	68.8%
Magnushammer (preprint 2023)	1,000 PISA theorems, Sledgehammer	38.3%
Same PISA evaluation	Magnushammer with learned premise selection	59.5%

- ITP 2022: 50 entries, 100 goals each; MePo, 512 facts in the base setting, 30 CPU seconds per configuration. Configurations are selected using the evaluation set. Reconstruction is not evaluated.
- PISA: Isabelle2021-1, Sledgehammer timeout 30 seconds, five local provers, union over several settings. Success requires a proof checked by Isabelle.
- Close dates do not make local goals and whole theorems, or search success and checked proofs, the same measure.

Mizar Evo: Design Principles

└ Backup 2. Sledgehammer Evaluations In Detail

└ Backup 2. Sledgehammer Evaluations In Detail

Backup 2. Sledgehammer Evaluations In Detail

Evaluation	Dataset and method	Success
ITP 2022	5,000 AFP goals, 16 greedy configurations	68.8%
Magnushammer (preprint 2023)	1,000 PISA theorems, Sledgehammer	38.3%
Same PISA evaluation	Magnushammer with learned premise selection	59.5%

- ITP 2022: 50 entries, 100 goals each; MePo, 512 facts in the base setting, 30 CPU seconds per configuration. Configurations are selected using the evaluation set. Reconstruction is not evaluated.
- PISA: Isabelle2021-1, Sledgehammer timeout 30 seconds, five local provers, union over several settings. Success requires a proof checked by Isabelle.
- Close dates do not make local goals and whole theorems, or search success and checked proofs, the same measure.

Read aloud:

- ITP 2022: 50 entries, 100 goals each; MePo, 512 facts in the base setting, 30 CPU seconds per configuration. Configurations are selected using the evaluation set. Reconstruction is not evaluated.
- PISA: Isabelle2021-1, Sledgehammer timeout 30 seconds, five local provers, union over several settings. Success requires a proof checked by Isabelle.
- Close dates do not make local goals and whole theorems, or search success and checked proofs, the same measure.

Optional notes and sources:

- Source: Desharnais et al., Seventeen Provers Under the Hammer, ITP 2022, sections 5, 5.6, Table 10; Mikuła et al., Magnushammer, arXiv:2303.04488 (preprint 2023), Table 2, Appendix A.4.

Backup 3. HOL-To-FOL Encodings And Reconstruction

- Function application: a variable function applied to an argument becomes $\text{app}(F, X)$; applications of constants may stay curried or be flattened.
- Lambda abstraction: lambda lifting introduces fresh constants with defining equations; combinator translation is the alternative.
- Types: polymorphic HOL types are encoded by guards, tags, or monomorphization; the choice affects soundness, completeness, and prover performance.
- Booleans: Boolean-valued terms inside formulas need a separate encoding.
- Reconstruction: `metis` with the used lemmas, `smt` replay of SMT proofs, or generated `Isar` text; evaluations count reconstruction failures separately.

Source: Meng and Paulson 2008; Blanchette, Böhme, Popescu, Smallbone 2016; Blanchette, Kaliszyk, Paulson, Urban 2016; Schurr, Fleury, Desharnais 2021.

Mizar Evo: Design Principles

└ Backup 3. HOL-To-FOL Encodings And Reconstruction

└ Backup 3. HOL-To-FOL Encodings And Reconstruction

Backup 3. HOL-To-FOL Encodings And Reconstruction

- Function application: a variable function applied to an argument becomes $\text{app}(F, X)$; applications of constants may stay curried or be flattened.
- Lambda abstraction: lambda lifting introduces fresh constants with defining equations; combinator translation is the alternative.
- Types: polymorphic HOL types are encoded by guards, tags, or monomorphization; the choice affects soundness, completeness, and prover performance.
- Booleans: Boolean-valued terms inside formulas need a separate encoding.
- Reconstruction: `metis` with the used lemmas, `smt` replay of SMT proofs, or generated `Isar` text; evaluations count reconstruction failures separately.

Source: Meng and Paulson 2008; Blanchette, Böhme, Popescu, Smallbone 2016; Blanchette, Kaliszyk, Paulson, Urban 2016; Schurr, Fleury, Desharnais 2021.

Read aloud:

- Function application: a variable function applied to an argument becomes $\text{app}(F, X)$; applications of constants may stay curried or be flattened.
- Lambda abstraction: lambda lifting introduces fresh constants with defining equations; combinator translation is the alternative.
- Types: polymorphic HOL types are encoded by guards, tags, or monomorphization; the choice affects soundness, completeness, and prover performance.
- Booleans: Boolean-valued terms inside formulas need a separate encoding.
- Reconstruction: `metis` with the used lemmas, `smt` replay of SMT proofs, or generated `Isar` text; evaluations count reconstruction failures separately.

Optional notes and sources:

- Source: Meng and Paulson 2008; Blanchette, Böhme, Popescu, Smallbone 2016; Blanchette, Kaliszyk, Paulson, Urban 2016; Schurr, Fleury, Desharnais 2021.

Backup 4. Template Instantiation: One Proof, Many Views

Bounded type parameter and generic theorem (specification example, proof omitted):

```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

Instantiation **specification example** (with the required registrations)

```
PermProduct[commutative associative unital AddMagma]
PermProduct[commutative associative unital MulMagma]
let R be commutative Ring; PermProduct[R qua AddMagma]  :: additive view
```

- A ring reaches Magma along two paths; qua selects the view, and the view also sets the notation.

Mizar Evo: Design Principles

└ Backup 4. Template Instantiation: One Proof, Many Views

└ Backup 4. Template Instantiation: One Proof, Many Views

Backup 4. Template Instantiation: One Proof, Many Views

Bounded type parameter and generic theorem (specification example, proof omitted):

```

definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T
      p being Permutation of dom s
      holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;

```

Instantiation **specification example** (with the required registrations)

```

PermProduct[commutative associative unital AddMagma]
PermProduct[commutative associative unital MulMagma]
let R be commutative Ring; PermProduct[R qua AddMagma] :: additive view

```

- A ring reaches Magma along two paths; qua selects the view, and the view also sets the notation.

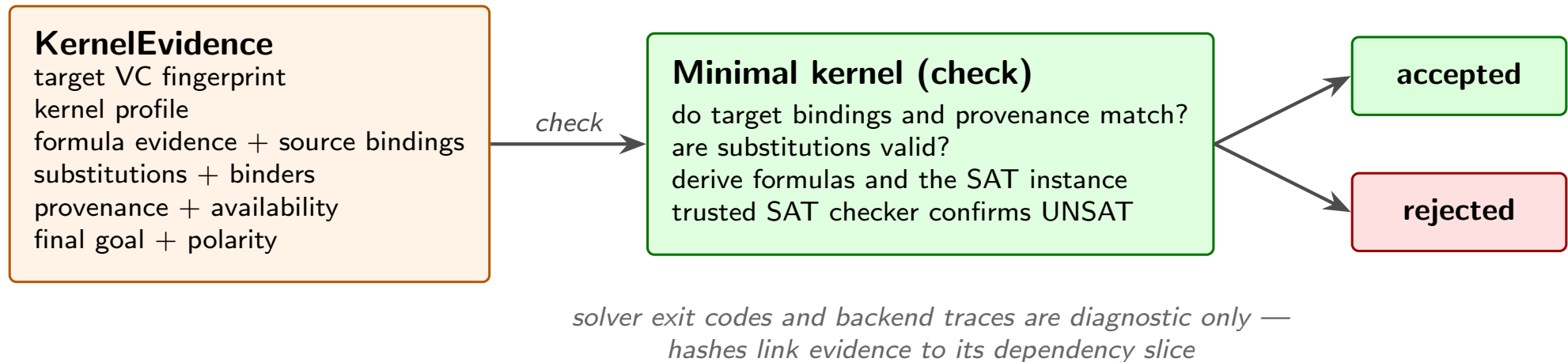
Read aloud:

- Bounded type parameter and generic theorem.
- A ring reaches Magma along two paths; qua selects the view, and the view also sets the notation.

Optional notes and sources:

- Source: doc/spec/en/18.templates.md, section 18.2.2.

Backup 5. Kernel Evidence And The Trusted SAT Check



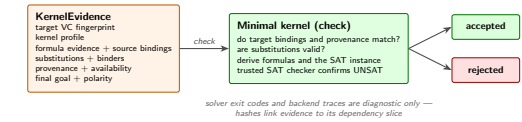
- The evidence holds source formulas, explicit substitutions, provenance, and target and goal bindings. The kernel checks them, builds a deterministic SAT problem, and requires UNSAT from its trusted in-process SAT checker.
- Backend proof traces, SMT proof objects, logs, and exit codes are for diagnosis only.

Mizar Evo: Design Principles

└ Backup 5. Kernel Evidence And The Trusted SAT Check

└ Backup 5. Kernel Evidence And The Trusted SAT Check

Backup 5. Kernel Evidence And The Trusted SAT Check



- The evidence holds source formulas, explicit substitutions, provenance, and target and goal bindings. The kernel checks them, builds a deterministic SAT problem, and requires UNSAT from its trusted in-process SAT checker.
- Backend proof traces, SMT proof objects, logs, and exit codes are for diagnosis only.

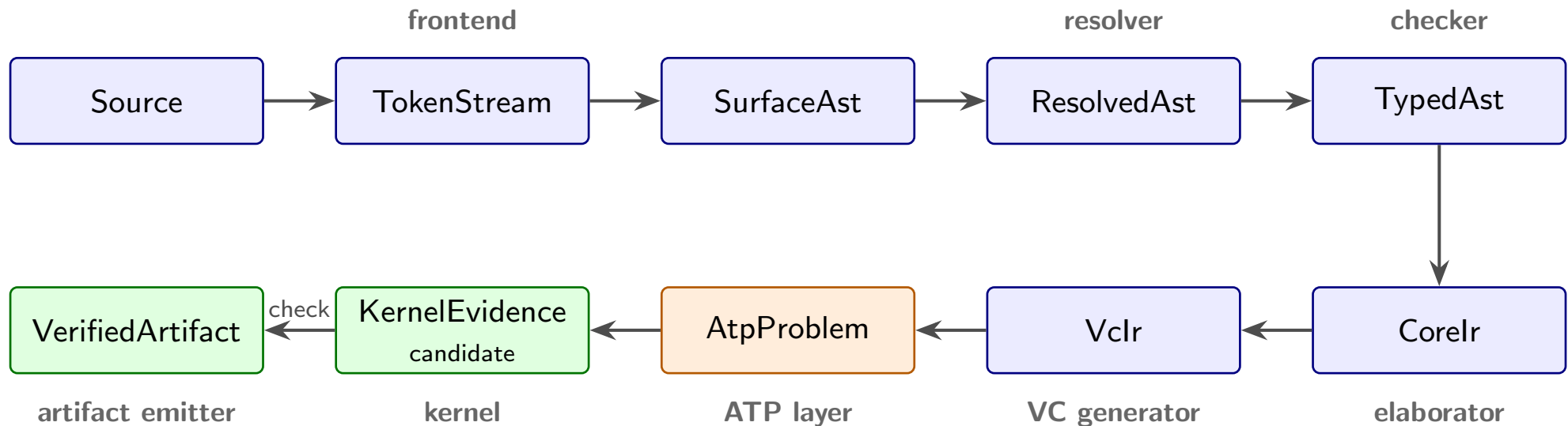
Read aloud:

- The evidence holds source formulas, explicit substitutions, provenance, and target and goal bindings. The kernel checks them, builds a deterministic SAT problem, and requires UNSAT from its trusted in-process SAT checker.
- Backend proof traces, SMT proof objects, logs, and exit codes are for diagnosis only.

Optional notes and sources:

- Source: `doc/design/architecture/en/08.reasoning_boundary.md`,
`15.kernel_certificate_format.md`.

Backup 6. The Core ATP Path



- Only obligations that are still open after deterministic discharge go to ATPs. The earlier discharge also needs replayable evidence.
- Each boundary states who owns a fact, which artifact records it, and what must be checked again after a change.

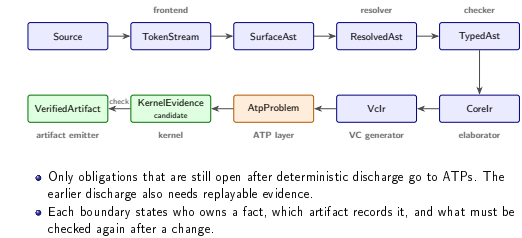
2026-10-09

Mizar Evo: Design Principles

└ Backup 6. The Core ATP Path

└ Backup 6. The Core ATP Path

Backup 6. The Core ATP Path



Read aloud:

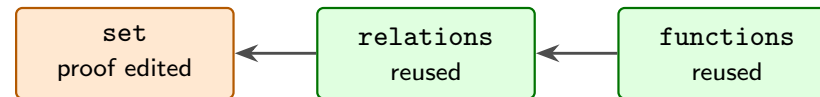
- Only obligations that are still open after deterministic discharge go to ATPs. The earlier discharge also needs replayable evidence.
- Each boundary states who owns a fact, which artifact records it, and what must be checked again after a change.

Optional notes and sources:

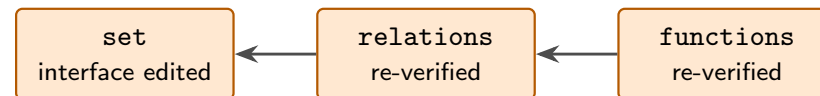
- Source: `doc/design/architecture/en/00.pipeline_overview.md`; Bialystok deck, Part 10.

Backup 7. Incremental Verification

theorem proof edit:
public statement + accepted status unchanged — importers reuse



interface change:
the dependency cone re-verifies



reuse requires all relevant keys: source/dependency slices, package/lockfile, toolchain/schema, registration/cluster, verifier policy, evidence/witnesses; missing data is a cache miss — a clean build must reproduce every acceptance

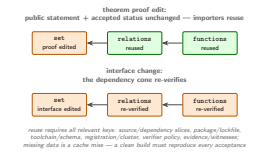
- A proof-body edit does not rebuild importers when the exported statement and the accepted status stay the same. An interface change re-verifies the dependency cone.
- All relevant cache keys must match; missing data is a cache miss. Cache reuse is never proof authority: a clean build must reproduce every acceptance.

Mizar Evo: Design Principles

└ Backup 7. Incremental Verification

└ Backup 7. Incremental Verification

Backup 7. Incremental Verification



- A proof-body edit does not rebuild importers when the exported statement and the accepted status stay the same. An interface change re-verifies the dependency cone.
- All relevant cache keys must match; missing data is a cache miss. Cache reuse is never proof authority: a clean build must reproduce every acceptance.

Read aloud:

- A proof-body edit does not rebuild importers when the exported statement and the accepted status stay the same. An interface change re-verifies the dependency cone.
- All relevant cache keys must match; missing data is a cache miss. Cache reuse is never proof authority: a clean build must reproduce every acceptance.

Optional notes and sources:

- Source: `doc/design/architecture/en/11.artifact_and_incremental_build.md`, `18.dependency_fingerprint.md`; Bialystok deck, Story 5.

Backup 8. Package Manifest

Mizar Evo **specification example**

```
[package]
name      = "algebra"
version   = "2.3.1"
edition   = "2025"

[dependencies]
mml_core  = "^1.0"
topology  = { version = "^0.9", features = ["metric"] }
```

- Reproducible builds need fixed source, lockfile, toolchain, and verifier settings, including deterministic ATP evidence.
- Versioned reuse replaces manual copying between article sets; an aggregate module can export a whole topic through one import.

Source: doc/spec/en/23.package_management_and_build_system.md; Bialystok deck, Story 1.

Mizar Evo: Design Principles

└ Backup 8. Package Manifest

└ Backup 8. Package Manifest

Backup 8. Package Manifest

Mizar Evo specification example

```
[package]
name = "algebra"
version = "2.3.1"
edition = "2025"

[dependencies]
mml_core = "1.0"
topology = { version = "0.9", features = ["metric"] }
```

- Reproducible builds need fixed source, lockfile, toolchain, and verifier settings, including deterministic ATP evidence.
- Versioned reuse replaces manual copying between article sets; an aggregate module can export a whole topic through one import.

Source: doc/spec/en/23.package_management_and_build_system.md; Bialystok deck, Story 1.

Read aloud:

- Reproducible builds need fixed source, lockfile, toolchain, and verifier settings, including deterministic ATP evidence.
- Versioned reuse replaces manual copying between article sets; an aggregate module can export a whole topic through one import.

Optional notes and sources:

- Source: doc/spec/en/23.package_management_and_build_system.md; Bialystok deck, Story 1.

Backup 9. Forty-Five-Minute Additions

Add selected examples where they fit in the story:

- ① HOL-to-FOL function encoding worked through on one goal (app, partial application, lambda lifting).
- ② Mizar function representation: `Function of X,Y, f.x`, soft type as a set-theoretic object.
- ③ Template: `PermProduct [T]` with additive and multiplicative views (Backup 4).
- ④ Algorithm: Euclidean GCD plus `by computation` and the promotion to a functor.
- ⑤ Structures and views: `AddMagma`, `MulMagma`, `Magma` (Bialystok Story 2).
- ⑥ Modern infrastructure: `mizar.pkg`, namespaces, the fingerprint graph (Backups 7-8).
- ⑦ Checking: evidence instantiation and the SAT check (Frame 6.3; details in Backup 5).

Mizar Evo: Design Principles

└ Backup 9. Forty-Five-Minute Additions

└ Backup 9. Forty-Five-Minute Additions

Backup 9. Forty-Five-Minute Additions

Add selected examples where they fit in the story:

- HOL-to-FOL function encoding worked through on one goal (app, partial application, lambda lifting).
- Mizar function representation: `Function of X,Y, f.x`, soft type as a set-theoretic object.
- Template: `PermProduct[T]` with additive and multiplicative views (Backup 4).
- Algorithm: Euclidean GCD plus by `computation` and the promotion to a functor.
- Structures and views: `AddMagma`, `MulMagma`, `Magma` (Bialystok Story 2).
- Modern infrastructure: `mizar.pkg`, namespaces, the fingerprint graph (Backups 7-8).
- Checking: evidence instantiation and the SAT check (Frame 6.3; details in Backup 5).

Read aloud:

- HOL-to-FOL function encoding worked through on one goal (app, partial application, lambda lifting).
- Mizar function representation: `Function of X,Y, f.x`, soft type as a set-theoretic object.
- Template: `PermProduct[T]` with additive and multiplicative views (Backup 4).
- Algorithm: Euclidean GCD plus by `computation` and the promotion to a functor.
- Structures and views: `AddMagma`, `MulMagma`, `Magma` (Bialystok Story 2).
- Modern infrastructure: `mizar.pkg`, namespaces, the fingerprint graph (Backups 7-8).
- Checking: evidence instantiation and the SAT check (Frame 6.3; details in Backup 5).

Backup 10. Sources And Attribution

Exact MML excerpts used in this talk:

Purpose	Source	Lines	Used in
Function application f.x	funct_1.miz	138-140	Frame 1.2
Function of X,Y as a soft type	funct_2.miz	87-90	Frame 1.2

Source URLs:

- FUNCT_1: https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz
- FUNCT_2: https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz

Attribution note:

- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.
- Benchmark figures: see Backups 1-2 and references.bib. Verify bibliographic metadata against publishers before the final deck.

Mizar Evo: Design Principles

└ Backup 10. Sources And Attribution

└ Backup 10. Sources And Attribution

Backup 10. Sources And Attribution

Exact MML excerpts used in this talk:

Purpose	Source	Lines	Used in
Function application $f.x$	<code>funct_1.miz</code>	138-140	Frame 1.2
Function of X,Y as a soft type	<code>funct_2.miz</code>	87-90	Frame 1.2

Source URLs:

- FUNCT_1: https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz
- FUNCT_2: https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz

Attribution note:

- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.
- Benchmark figures: see Backups 1-2 and `references.bib`. Verify bibliographic metadata against publishers before the final deck.

Read aloud:

- FUNCT_1: https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz
- FUNCT_2: https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz
- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.
- Benchmark figures: see Backups 1-2 and `references.bib`. Verify bibliographic metadata against publishers before the final deck.

Backup 11. Comparing Evaluation Conditions

	MizAR 60 (ITP 2023)	Sledgehammer / AFP (ITP 2022)
success	58.4% (1,690 / 2,896)	68.8% (3,440 / 5,000)
counts	MML 1147: 2,896 holdout theorems/lemmas (57,897 total)	5,000 local goals; 50 AFP entries
premises	learned selection from the whole library	MePo; base 512 facts, varied by configuration
budget	portfolio: 420 CPU s	16 greedy configurations, 30 CPU s each: 480 CPU s
status	ATP proof found (hammering)	external proof found; no Isabelle reconstruction

- Different units, premises, budgets, and configurations; the rates do not rank the designs.

Mizar Evo: Design Principles

└ Backup 11. Comparing Evaluation Conditions

└ Backup 11. Comparing Evaluation Conditions

Backup 11. Comparing Evaluation Conditions

	Mizar 60 (ITP 2023)	Sledgehammer / AFP (ITP 2022)
success	58.4% (1,690 / 2,896)	68.8% (3,440 / 5,000)
counts	MML 1147; 2,896 holdings theorems/kmmas (57,897 total)	5,000 local goals; 50 AFP entries
premises	learned selection from the whole library	MePo: base 512 facts, varied by configuration
budget	portfolio: 420 CPU s	10 greedy configurations, 30 CPU s each; 480 CPU s
status	ATP proof found (hammering)	external proof found; no Isabelle reconstruction

• Different units, premises, budgets, and configurations; the rates do not rank the designs.

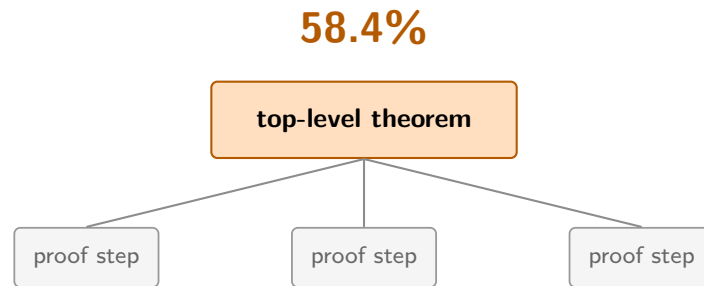
Read aloud:

- **Different units, premises, budgets, and configurations; the rates do not rank the designs.**

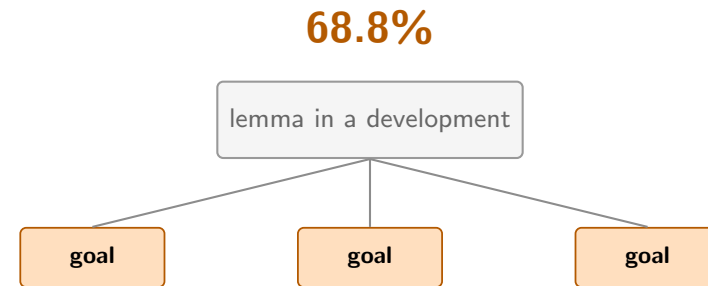
Optional notes and sources:

- A "hammer" is a tool that sends a goal to automatic provers. "Premises" are the facts the prover may use.
- Source: Jakubův et al. 2023, sections 6.2, 6.5; Desharnais et al. 2022, sections 5, 5.6, Table 10. The greedy configurations were selected using the evaluation set itself.
- Publication dates differ from experiment dates. MizAR's main experiments ran in 2020-2021; the AFP study uses Isabelle from January 2022 and AFP from December 2021.
- MizAR's 75% figure uses premises chosen from the library by a human or a machine. Keep it for questions.

Backup 12. What Each Number Counts



Mizar 60 (ITP 2023) counts *whole top-level theorems and lemmas*
2,896 holdout problems from 57,897 in
MML 1147; learned premises; 420 CPU s



Sledgehammer / AFP (ITP 2022)
counts *proof goals as they arise*
5,000 goals from 50 entries; greedy 16 con-
figurations × 30 CPU s; no reconstruction

Different units, premise selection, and budgets.

- MizAR asks: can the machine prove this whole theorem from the library, with no help from the author?
- The AFP study asks: can the machine close this goal where it appears, often inside a proof that a person has already written?
- Both are fair questions. They are not the same question.

2026-10-09

Mizar Evo: Design Principles

└ Backup 12. What Each Number Counts

└ Backup 12. What Each Number Counts

Backup 12. What Each Number Counts



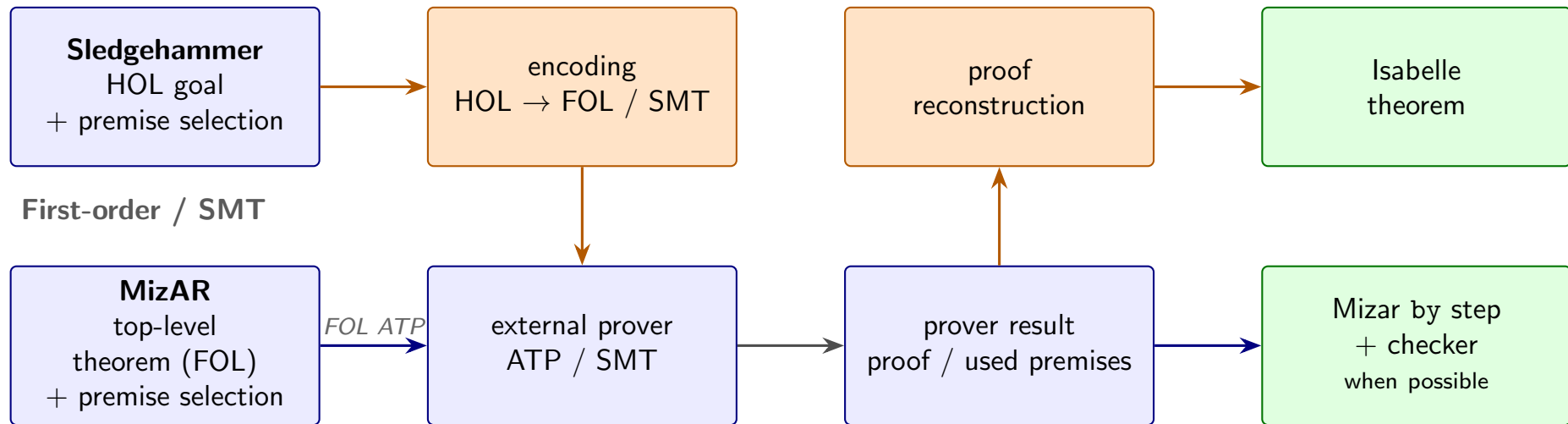
- MizAR asks: can the machine prove this whole theorem from the library, with no help from the author?
- The AFP study asks: can the machine close this goal where it appears, often inside a proof that a person has already written?
- Both are fair questions. They are not the same question.

Read aloud:

- **MizAR asks: can the machine prove this whole theorem from the library, with no help from the author?**
- **The AFP study asks: can the machine close this goal where it appears, often inside a proof that a person has already written?**
- Both are fair questions. They are not the same question.

Backup 13. Two Paths To A First-Order Prover

Higher-order (Isabelle/HOL)



Mizar Evo: kernel evidence checking replaces the last step (later slide).

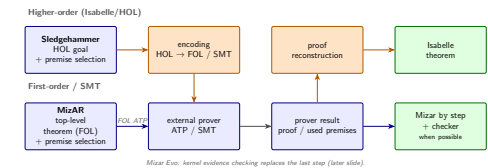
- In Sledgehammer's first-order and SMT path, a higher-order goal is translated, and the proof is rebuilt inside Isabelle.
- MizAR gives the prover a problem that is already first-order. The generated problem is first-order.

Mizar Evo: Design Principles

└ Backup 13. Two Paths To A First-Order Prover

└ Backup 13. Two Paths To A First-Order Prover

Backup 13. Two Paths To A First-Order Prover



- In Sledgehammer's first-order and SMT path, a higher-order goal is translated, and the proof is rebuilt inside Isabelle.
- Mizar gives the prover a problem that is already first-order. The generated problem is first-order.

Read aloud:

- In Sledgehammer's first-order and SMT path, a higher-order goal is translated, and the proof is rebuilt inside Isabelle.
- Mizar gives the prover a problem that is already first-order. The generated problem is first-order.

Optional notes and sources:

- Both paths describe existing systems (Blanchette, Kaliszyk, Paulson, Urban 2016; Jakubův et al. 2023). The orange boxes are the translation and reconstruction layers.
- The figure shows the first-order and SMT path. The ITP 2022 evaluation also includes provers using native higher-order formats.
- Mizar's headline percentages count ATP proofs. The Mizar checker re-checks the resulting by step when the inference is within its strength.

Backup 14. Functions In Higher-Order Logic

Schematic HOL statement **sketch** (Isabelle/HOL-style notation)

```
f :: 'a => 'b      x :: 'a
```

```
P (f x)
```

- In HOL, function types and function application are part of the logic itself.
- Functions as arguments, functions that return functions, partial application, lambda: all can be written directly.
- This is very convenient for the person who writes mathematics.
- But E and Vampire are first-order provers. The higher-order structure must be encoded before they can see it.

Mizar Evo: Design Principles

└ Backup 14. Functions In Higher-Order Logic

└ Backup 14. Functions In Higher-Order Logic

Backup 14. Functions In Higher-Order Logic

Schematic HOL statement **sketch** (Isabelle/HOL-style notation) $f :: 'a \rightarrow 'b$ $x :: 'a$ $P (f\ x)$

- In HOL, function types and function application are part of the logic itself.
- Functions as arguments, functions that return functions, partial application, lambda: all can be written directly.
- This is very convenient for the person who writes mathematics.
- But E and Vampire are first-order provers. The higher-order structure must be encoded before they can see it.

Read aloud:

- Schematic HOL statement.
- **In HOL, function types and function application are part of the logic itself.**
- Functions as arguments, functions that return functions, partial application, lambda: all can be written directly.
- **This is very convenient for the person who writes mathematics.**
- **But E and Vampire are first-order provers. The higher-order structure must be encoded before they can see it.**

Backup 15. HOL Encoding And Reconstruction

Encoding steps, schematic **sketch**

<code>F X</code>	<code>-> app(F, X)</code>	
<code>(%x. t) ...</code>	<code>-> fresh constant + defining axioms</code>	<code>(lambda lifting)</code>
<code>polymorphic types</code>	<code>-> type guards or type tags</code>	
<code>Boolean-valued terms</code>	<code>-> extra encoding</code>	

- Applying a variable function becomes an explicit `app` symbol. Lambdas are lifted out or turned into combinators. Types become guards or tags.
- After the prover succeeds, the proof must come back into Isabelle: a `metis` call, an `smt` replay, or generated `Isar` text. This step is called reconstruction.
- These mechanisms connect HOL goals to first-order provers.
- This describes the connection, not a measured performance ranking.

Mizar Evo: Design Principles

└ Backup 15. HOL Encoding And Reconstruction

└ Backup 15. HOL Encoding And Reconstruction

Backup 15. HOL Encoding And Reconstruction

Encoding steps, schematic **sketch**

$\lambda x. t$	$\rightarrow$	<code>app(F, X)</code>
$(\lambda x. t) \dots$	$\rightarrow$	fresh constant + defining axioms (lambda lifting)
polymorphic types	$\rightarrow$	type guards or type tags
Boolean-valued terms	$\rightarrow$	extra encoding

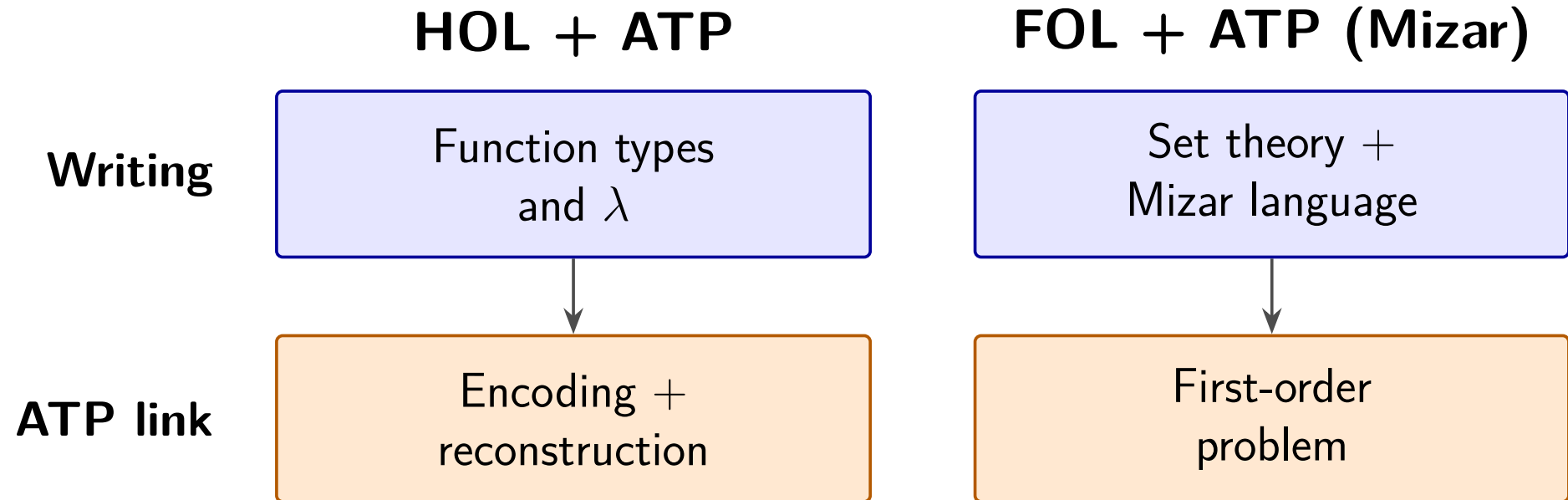
- Applying a variable function becomes an explicit app symbol. Lambdas are lifted out or turned into combinators. Types become guards or tags.
- After the prover succeeds, the proof must come back into Isabelle: a `metis` call, an `smt` replay, or generated `Isar` text. This step is called reconstruction.
- These mechanisms connect HOL goals to first-order provers.
- This describes the connection, not a measured performance ranking.

Read aloud:

- Encoding steps, schematic.
- **Applying a variable function becomes an explicit app symbol. Lambdas are lifted out or turned into combinators. Types become guards or tags.**
- **After the prover succeeds, the proof must come back into Isabelle: a `metis` call, an `smt` replay, or generated `Isar` text. This step is called reconstruction.**
- These mechanisms connect HOL goals to first-order provers.
- This describes the connection, not a measured performance ranking.

Optional notes and sources:

- Source: Meng and Paulson 2008 (translating higher-order clauses to first-order clauses); Blanchette, Böhme, Popescu, Smallbone 2016 (type encodings); Blanchette, Kaliszyk, Paulson, Urban 2016 (survey); Schurr, Fleury, Desharnais 2021 (reconstruction).
- The exact encodings that Sledgehammer uses depend on the prover and the options. This frame is schematic.



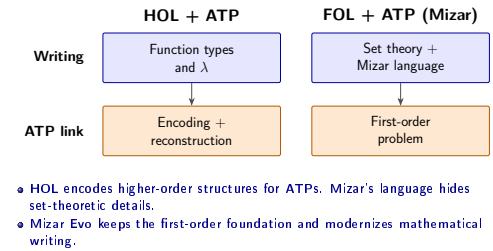
- HOL encodes higher-order structures for ATPs. Mizar's language hides set-theoretic details.
- Mizar Evo keeps the first-order foundation and modernizes mathematical writing.

Mizar Evo: Design Principles

└ Backup 16. Where Complexity Lives

└ Backup 16. Where Complexity Lives

Backup 16. Where Complexity Lives



Read aloud:

- **HOL encodes higher-order structures for ATPs. Mizar's language hides set-theoretic details.**
- **Mizar Evo keeps the first-order foundation and modernizes mathematical writing.**

Optional notes and sources:

- HOL and FOL put complexity in different layers. HOL has function types and lambdas in its logic; its first-order ATP path uses encoding and reconstruction.
- Mizar's soft types, modes, attributes, registrations, and schemes hide details such as pairs and domains that appear in raw set-theoretic writing.

Backup 17. Design Trade-Offs

	HOL ITP + ATP	FOL ITP + ATP
writing	higher-order features, short text	long text if written raw
functions	basic higher-order objects	first-order set-theoretic objects
ATP connection	HOL-to-FOL / SMT encoding	generation of a first-order problem
reconstruction	reconstruction in Isabelle	rechecking Mizar steps where supported
the language must provide	HOL abstraction	abstraction that hides first-order details

- This table compares representations and checking paths. It gives no measured performance advantage.

Mizar Evo: Design Principles

└ Backup 17. Design Trade-Offs

└ Backup 17. Design Trade-Offs

Backup 17. Design Trade-Offs

	HOL ITP + ATP	FOL ITP + ATP
writing	higher-order features, short text	long text if written raw
functions	basic higher-order objects	first-order set-theoretic objects
ATP connection	HOL-to-FOL / SMT encoding	generation of a first-order problem
reconstruction	reconstruction in Isabelle	rechecking Mizar steps when supported
the language must provide	HOL abstraction	abstraction that hides first-order details

- This table compares representations and checking paths. It gives no measured performance advantage.

Read aloud:

- This table compares representations and checking paths. It gives no measured performance advantage.